UNIVERSIDAD NACIONAL DE LOJA

FACULTAD DE LA ENERGÍA, LAS INDUSTRIAS Y LOS RECURSOS NATURALES NO RENOVABLES

Estudiante: Danny Fernando Beltrán

Carrera: Computación

Docente: Ing. Cristian Narvaez

Asignatura: Teoría de la distribución y probabilidad

Unidad: Uno

Período académico

marzo 2026 – agosto 2026

Persistencia “LocalStorage”

Usar bases de datos es la mejor forma y más segura de persistir información. Pero también tenemos una alternativa más sencilla y rápida llamada “LocalStorage”.

La persistencia de datos es la manera de conservar información para utilizarla posteriormente, hasta cuando cerremos o actualicemos un programa.

Las variables de JavaScript están en la memoria durante la ejecución y sus valores pueden perderse al recargar la página, justo en este momento es donde interviene “localStorage”, una herramienta integrada en los navegadores que permite guardar datos de manera sencilla sin necesidad de crear una base de datos en un servidor [1].

¿Cómo funciona LocalStorage?

LocalStorage funciona almacenando información en el navegador mediante pares de clave y valor. Por ejemplo, se puede guardar el nombre de una persona y recuperarlo posteriormente cuando sea necesario.

Los principales métodos son:

- localStorage.setItem(): guarda un dato.
- localStorage.getItem(): recupera un dato guardado.
- JSON.stringify(): convierte un objeto o arreglo de JavaScript en una cadena de texto JSON.
- JSON.parse(): convierte una cadena JSON nuevamente en un objeto o arreglo de JavaScript.

Es de vital importancia que tomemos en cuenta que localStorage almacena cadenas de texto. Por este motivo, para conservar objetos y arreglos correctamente, es necesario convertirlos a JSON antes de guardarlos.

Diferencias entre Local Storage y Session Storage

En el *local storage* la información permanece disponible siempre aun cuando la pestaña o la ventana del navegador se cierra, pero la borra cuando la información está almacenada en *session storage,* se puede accedera la información almacenada en el *local storage* desde múltiples pestañas o ventanas del navegador, por otro lado, solo se puede acceder a la información almacenada en *session storage* desde pestañas o ventanas específicas del navegador porque no se comparte [2]

Cómo usar el Local Storage

El objeto *local storage* ya integra varios métodos diferentes para poder interactuar con él. Con estos métodos su puede agregar, leer, y borrar información que se encuentra en el l*ocal storage*.

Para almacenar información en Local Storage se usa en el método setItem(). Este método lleva dos argumentos, un clave y un valor.

Para leer información almacenada en el Local Storage se utiliza el método getItem(). Este método lleva una clave como argumento.

Para almacenar y leer valores de información complejoa en el Local Storage se tiene que almacenar en cadenas de texto. Por esta razón, si necesitamos almacenar valores como objetos o *arrays*, lo primero que se tiene que hacer es cambiar ese valor en una cadena de texto. Esto se lo puede hacer usando el método JSON.stringify().

Si se necesita acceder a la información del *local storage*, tenemos que devolverla a su forma original. Esto lo podemos realizar usando el método JSON.parse()

Para borrar información utilizamos removeItem() se usa para borrar un solo item o también clear() que se usa para borrar toda la información,

Para obtener el nombre de una clave se usa: key().

Ejemplo 1: Guardar y mostrar un nombre

```html
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Ejemplo 1</title>
</head>
<body>
<h2>Ejemplo 1: Guardar un nombre</h2>

<script>
    // Guardamos el nombre en localStorage
    localStorage.setItem("nombre", "Carlos");

    // Recuperamos el nombre guardado
    let nombre = localStorage.getItem("nombre");

    // Mostramos el nombre en la página
    document.write("El nombre guardado es: " + nombre);
</script>
</body>
</html>
```

Explicación: setItem() guarda el nombre y getItem() lo recupera para mostrarlo en la página.

Ejemplo 2: Guardar un color favorito

```html
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Ejemplo 2</title>
</head>
<body>
<h2>Ejemplo 2: Color favorito</h2>

<script>
    // Guardamos el color favorito
    localStorage.setItem("color", "azul");

    // Recuperamos el color guardado
    let color = localStorage.getItem("color");

    // Mostramos el color en la página
    document.write("Mi color favorito es: " + color);
</script>
</body>
</html>
```

Explicación: Se guarda la preferencia de color para poder recuperarla después, incluso al volver a abrir la página.

Ejemplo 3: Guardar y eliminar una edad



```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Ejemplo 3</title>
</head>
<body>
<h2>Ejemplo 3: Eliminar una edad</h2>

<script>
    // Guardamos la edad
    localStorage.setItem("edad", "20");

    // Recuperamos la edad
    let edad = localStorage.getItem("edad");

    // Mostramos la edad antes de eliminarla
    document.write("Edad guardada: " + edad + "<br>");

    // Eliminamos la edad almacenada
    localStorage.removeItem("edad");

    // Comprobamos si el dato todavía existe
    let edadEliminada = localStorage.getItem("edad");

    // Mostramos el resultado
    document.write("Edad después de eliminarla: " + edadEliminada);
</script>
</body>
</html>
```

Explicación: removeItem() elimina la edad guardada. Después, getItem() devuelve null porque el dato ya no existe.

Explique cómo la latencia de almacenamiento influye en la velocidad de respuesta del software.

La latencia de almacenamiento es el tiempo que se demora un sistema para completar una operación de lectura o escritura de datos. Interviene en la velocidad de respuesta del software porque, cuando un programa tiene que acceder a la información guardada en su disco o en una base de datos tiene que espera a que esa operación finalice para que pueda continuar.

En ejemplo, el de un producto que es buscado por un cliente, puede tarda un rato hasta localizar esa información en una base de datos, pero si se usara una memoria caché se lograría hacer esta tarea en mucho menor tiempo.

A si mismo, cuando los datos o información se fragmenta puede afectar el rendimiento, especialmente en discos duros mecánicos, porque acceder a información distribuida en diferentes lugares puede requerir más movimientos y operaciones de lectura. Para reducir estos retrasos, se utiliza índices, caché, organizar adecuadamente las estructuras de datos y agrupar las operaciones de entrada y salida (I/O).

Conclusiones:

Para concluir, puedo determinar que la persistencia de datos en una base de datos es fundamental para conservar y proteger la información, permitiendo acceder a ella y utilizarla cuando sea necesario.

Para terminar, mientras mayor sea la latencia de almacenamiento, mayor puede ser el tiempo de respuesta del software. Por eso, elegir un almacenamiento adecuado y reducir las operaciones innecesarias de lectura y escritura permite desarrollar aplicaciones más rápidas y eficientes.

Declaración de IA

Se utilizó ChatGPT como herramienta de apoyo para comprender los ejemplos de las fuentes consultadas y aclarar conceptos técnicos. Asimismo, se empleó para mejorar la redacción y la coherencia del contenido elaborado. La información fue revisada para asegurar su comprensión.

Referencias

[1] M. Burgos., «DEV Community.,» 2022. [En línea]. Available: https://dev.to/maxwellnewage/persistencia-de-datos-en-javascript-localstorage-419m.

[2] B. Semah., «freeCodeCamp,» 2024. [En línea]. Available: https://www.freecodecamp.org/espanol/news/como-usar-localstorage-en-javascript/.